

Assignment No: 1(B)

```
#include <iostream>
#include <vector>
#include <omp.h>

using namespace std;

class Graph {
    int V;
    vector<vector<int>> adj;

public:
    Graph(int V) {
        this->V = V;
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);    // undirected graph
    }

    void DFSUtil(int node, vector<bool> &visited) {

        visited[node] = true;

        #pragma omp critical
        cout << node << " ";

        #pragma omp parallel for
        for(int i = 0; i < adj[node].size(); i++) {

            int neighbor = adj[node][i];

            if(!visited[neighbor]) {
                DFSUtil(neighbor, visited);
            }
        }
    }

    void parallelDFS(int start) {

        vector<bool> visited(V, false);

        cout << "\nDFS Traversal Order: ";
```

```

        DFSUtil(start, visited);

        cout << endl;
    }
};

int main() {

    int V, E;

    cout << "Enter number of vertices: ";
    cin >> V;

    Graph g(V);

    cout << "Enter number of edges: ";
    cin >> E;

    cout << "Enter edges (u v):\n";

    for(int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        g.addEdge(u, v);
    }

    int start;

    cout << "Enter starting vertex: ";
    cin >> start;

    g.parallelDFS(start);

    return 0;
}

```

Input:

Enter number of vertices: 5

Enter number of edges: 4

Enter edges (u v):

0 1

0 2

1 3

1 4

Enter starting vertex: 0

Output:

Parallel BFS Traversal: 0 1 3 4 2